

УНИВЕРСИТЕТ ИТМО

Факультет программной инженерии и компьютерной техники

Направление подготовки 09.03.04 Программная инженерия

Дисциплина «Базы данных»

Лабораторная работа №4

Студент: Попов Глеб Ильич

Практик: Райла Мартин

Санкт-Петербург, 2025 год

Задание:

Введите вариант:

Внимание! У разных вариантов разный текст задания!

Составить запросы на языке SQL (пункты 1-2).

Для каждого запроса предложить индексы, добавление которых уменьшит время выполнения запроса (указать таблицы/атрибуты, для которых нужно добавить индексы, написать тип индекса; объяснить, почему добавление индекса будет полезным для данного запроса).

Для запросов 1-2 необходимо составить возможные планы выполнения запросов. Планы составляются на основании предположения, что в таблицах отсутствуют индексы. Из составленных планов необходимо выбрать оптимальный и объяснить свой выбор. Изменятся ли планы при добавлении индекса и как?

Для запросов 1-2 необходимо добавить в отчет вывод команды EXPLAIN ANALYZE [запрос]

Подробные ответы на все вышеперечисленные вопросы должны присутствовать в отчете (планы выполнения запросов должны быть нарисованы, ответы на вопросы - представлены в текстовом виде).

1. Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:
Таблицы: Н_ЛЮДИ, Н_ВЕДОМОСТИ.
Вывести атрибуты: Н_ЛЮДИ.ИД, Н_ВЕДОМОСТИ.ИД.
Фильтры (AND):
а) Н_ЛЮДИ.ИД = 163484.
б) Н_ВЕДОМОСТИ.ДАТА > 2010-06-18.
в) Н_ВЕДОМОСТИ.ДАТА = 2010-06-18.
Вид соединения: INNER JOIN.
2. Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:
Таблицы: Н_ЛЮДИ, Н_ВЕДОМОСТИ, Н_СЕССИЯ.
Вывести атрибуты: Н_ЛЮДИ.ИД, Н_ВЕДОМОСТИ.ИД, Н_СЕССИЯ.ДАТА.
Фильтры (AND):
а) Н_ЛЮДИ.ОТЧЕСТВО < Владимирович.
б) Н_ВЕДОМОСТИ.ДАТА < 2010-06-18.
Вид соединения: INNER JOIN.

Выполнение:

Запрос 1.

```
SELECT Н_люди.ИД, Н_ВЕДОМОСТИ.ИД
FROM Н_люди
INNER JOIN Н_ВЕДОМОСТИ
  ON Н_люди.ИД = Н_ВЕДОМОСТИ.ЧЛВК_ИД
WHERE
  Н_люди.ИД = 163484 AND Н_ВЕДОМОСТИ.ДАТА > '2010-06-18' AND Н_ВЕДОМОСТИ.ДАТА = '2010-06-18';
```

Использование индексов.

Индекс B-tree на атрибуте ИД уменьшит время выполнения запроса, потому что индекс B-tree хорошо подходит для операций(=, <=, >, >=, <, <=) и joinax . Например, в 1 запросе есть операция выборки:(where Н_ЛЮДИ.ИД = ...,). Такой индекс позволит нам быстро находить строки с заданным значением ИД людей, и исчезнет необходимость полного перебора таблицы. Это повысит эффективность поиска определенного ИД в таблице с большим количеством людей.

Индекс B-tree на атрибутах ЧЛВК_ИД и ДАТА уменьшит время выполнения запроса, потому что индекс B-tree хорошо подходит для операций(=, <=, >, >=, <, <=) и joinax. В запросе 1 соединение выполняется по полю ЧЛВК_ИД, а фильтрация — по полю ДАТА (WHERE Н_ВЕДОМОСТИ.ДАТА > ... AND Н_ВЕДОМОСТИ.ДАТА = ...). Такой B-tree индекс позволит быстро находить все записи конкретного человека и сразу же ограничивать их по дате, что значительно повысит производительность при больших объемах данных.

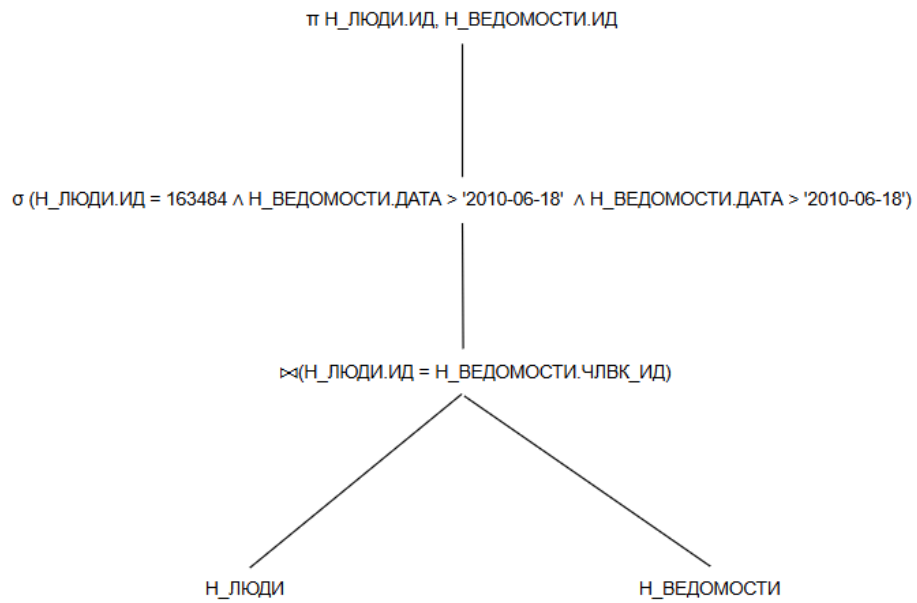
Hash-index будет эффективен при запросах по равенству без дополнительных условий. Однако в запросе 1 помимо условия равенства по ИД есть два дополнительных условия, что делает hash-index неэффективным в нашем случае.

Индекс GiST будет неэффективным так как предназначен для индексирования сложных структур данных и не применяется для поиска по числовым и строковым атрибутам

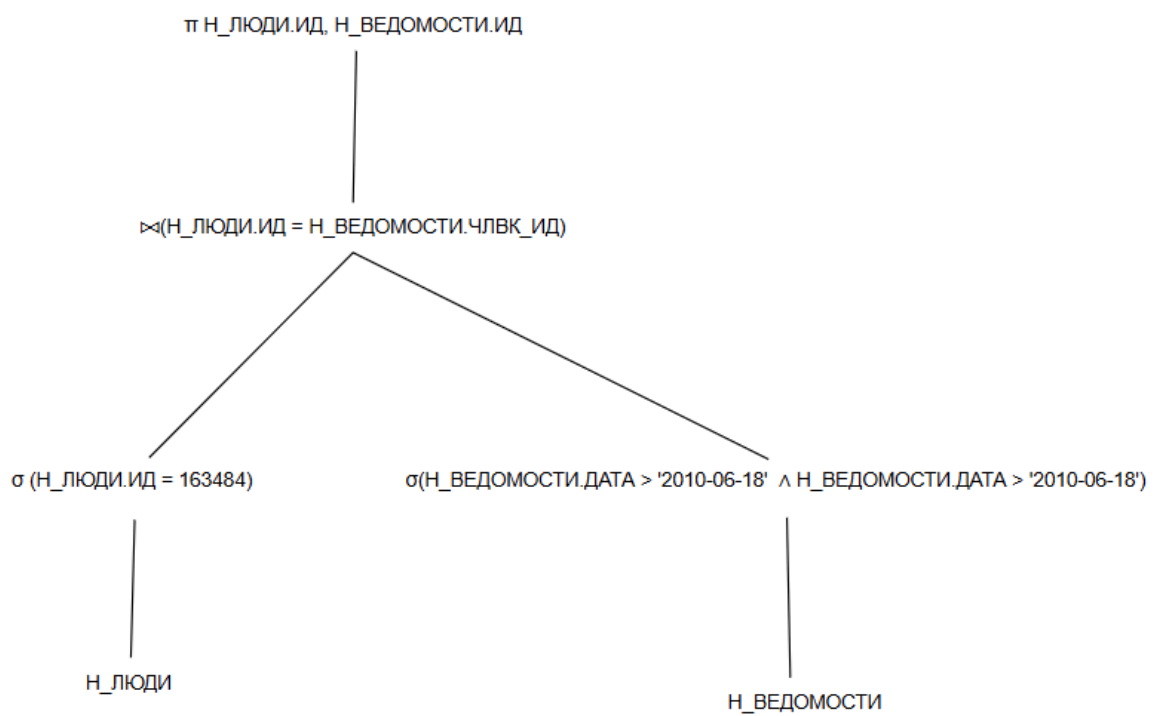
Индекс GIN также неэффективен, так как он используется для полнотекстового поиска, работы с массивами и JSON-полями, а не для поиска по обычным строковым и числовым столбцам

План выполнения запроса 1:

План 1:



План 2:



Для первого запроса есть два возможных плана выполнения: с фильтрацией после соединения (план 1) и с фильтрацией до соединения (план 2). Оптимальным является второй вариант, поскольку фильтрация до соединения существенно уменьшает объем данных, участвующих в операциях соединения, и ускоряет выполнение запроса.

После добавления индексов план выполнения запроса изменится: СУБД начнет использовать B-tree индексы по полям Н_ЛЮДИ.ИД и Н_ВЕДОМОСТИ.ДАТА. Это позволит сразу выбирать только те строки из таблицы Н_ЛЮДИ, у которых ИД равен 163484, и только те строки из таблицы Н_ВЕДОМОСТИ, у которых дата больше '2010-06-18' и равна '2010-06-18', без необходимости последовательного просмотра всей таблицы. Таким образом, индексы позволяют реализовать оптимизированный план максимально эффективно, обеспечивая быстрый доступ к нужным данным и ускоряя выполнение запроса.

Выполнение команды EXPLAIN ANALYZE

```
QUERY PLAN
-----
Nested Loop (cost=9.70..17.75 rows=1 width=8) (actual time=0.038..0.040 rows=0 loops=1)
-> Index Only Scan using "ЧЛВК_ПК" on "Н_ЛЮДИ" (cost=0.28..4.30 rows=1 width=4) (actual time=0.028..0.029 rows=1 loops=1)
    Index Cond: ("ИД" = 163484)
    Heap Fetches: 0
-> Bitmap Heap Scan on "Н_ВЕДОМОСТИ" (cost=9.42..13.44 rows=1 width=8) (actual time=0.004..0.005 rows=0 loops=1)
    Recheck Cond: (("ДАТА" > '2010-06-18 00:00:00'::timestamp without time zone) AND ("ДАТА" = '2010-06-18 00:00:00'::timestamp without time zone) AND ("ЧЛВК_ИД" = 163484))
-> BitmapAnd (cost=9.42..9.42 rows=1 width=0) (actual time=0.002..0.003 rows=0 loops=1)
    -> Bitmap Index Scan on "ВЕД_ДАТА_I" (cost=0.00..4.38 rows=9 width=0) (actual time=0.001..0.002 rows=0 loops=1)
        Index Cond: (("ДАТА" > '2010-06-18 00:00:00'::timestamp without time zone) AND ("ДАТА" = '2010-06-18 00:00:00'::timestamp without time zone))
    -> Bitmap Index Scan on "ВЕД_ЧЛВК_FK_IFK" (cost=0.00..4.78 rows=65 width=0) (never executed)
        Index Cond: ("ЧЛВК_ИД" = 163484)
Planning Time: 1.040 ms
Execution Time: 0.142 ms
(13 строк)
```

Запрос 2:

```

SELECT Н_ЛЮДИ.ИД, Н_ВЕДОМОСТИ.ИД, Н_СЕССИЯ.ДАТА
FROM Н_ЛЮДИ
JOIN Н_ВЕДОМОСТИ |
    ON Н_ВЕДОМОСТИ.ЧЛВК_ИД = Н_ЛЮДИ.ИД
JOIN Н_СЕССИЯ
    ON Н_СЕССИЯ.ЧЛВК_ИД = Н_ЛЮДИ.ИД
WHERE Н_ЛЮДИ.ОТЧЕСТВО < 'Владимирович' AND Н_ВЕДОМОСТИ.ДАТА < '2010-06-18';

```

Индекс B-tree на атрибуте ОТЧЕСТВО будет эффективен и уменьшит время выполнения запроса, потому что индекс B-tree хорошо подходит для операций (=, с =, >, >=, <, <=) и joinax и ускорит выборку по диапазону (WHERE Н_ЛЮДИ.ОТЧЕСТВО < ...). Такой индекс позволит быстро находить всех людей, чьи отчества меньше заданного значения, без необходимости полного перебора таблицы.

Индекс B-tree на атрибутах ЧЛВК_ИД и ДАТА уменьшит время выполнения запроса, потому что соединение выполняется по ЧЛВК_ИД, а фильтрация по ДАТА < Индекс B-tree хорошо подходит для операций (=, с =, >, >=, <, <=) и joinax и позволит быстро выбирать все ведомости для конкретного человека и сразу ограничивать их по дате, ускоряя выполнение запроса.

Индекс B-tree на атрибуте ЧЛВК_ИД уменьшит время выполнения запроса, так как соединение с этой таблицей выполняется по этому полю (JOIN Н_СЕССИЯ ON Н_СЕССИЯ.ЧЛВК_ИД = Н_ЛЮДИ.ИД). Такой индекс позволит быстро находить все сессии для нужного человека так как индекс B-tree хорошо подходит для операций (=, с =, >, >=, <, <=) и joinax

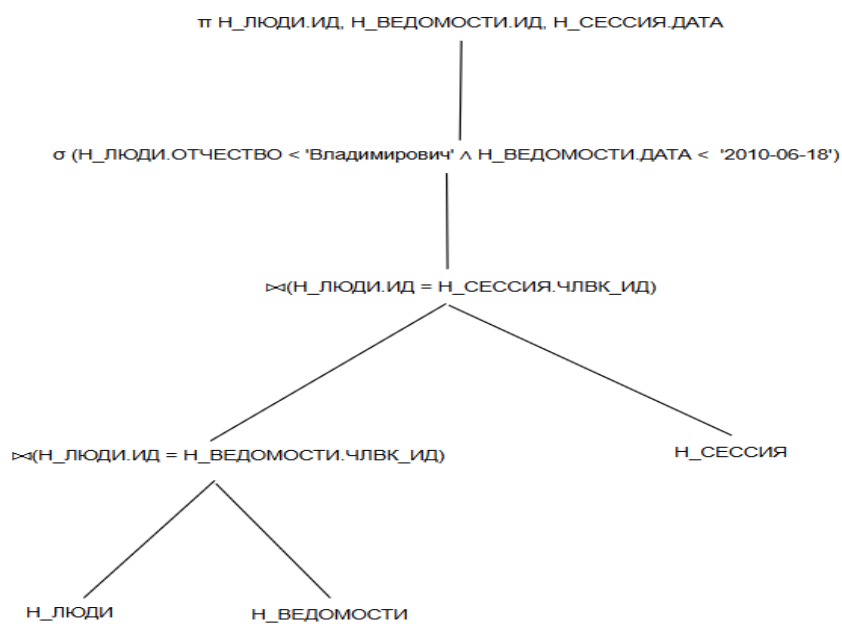
Hash-index будет эффективен при запросах по равенству без дополнительных условий. Однако в запросе 2 отсутствуют условия на равенство(в запросе 2 условия <), что делает hash-index неэффективным в нашем случае.

Индекс GiST будет неэффективным так как предназначен для индексирования сложных структур данных и не применяется для поиска по числовым и строковым атрибутам

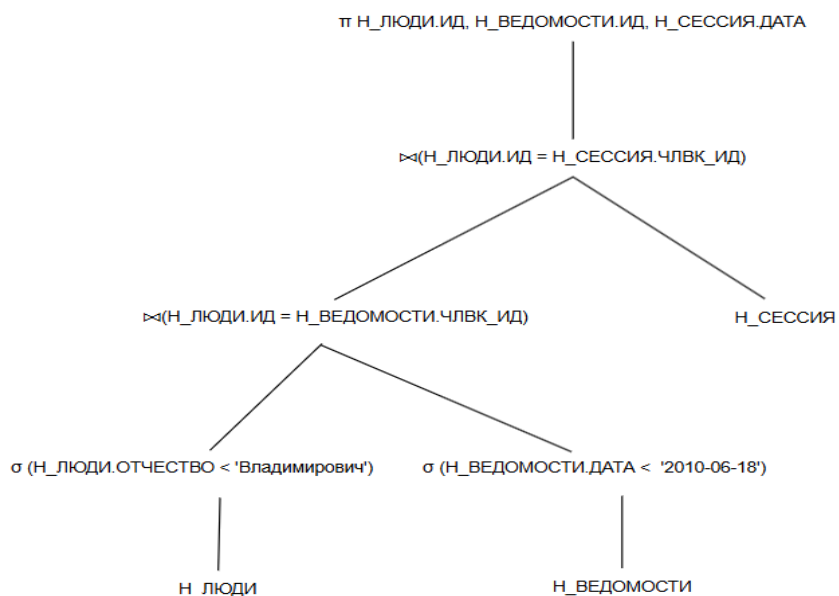
Индекс GIN также неэффективен, так как он используется для полнотекстового поиска, работы с массивами и JSON-полями, а не для поиска по обычным строковым и числовым столбцам

План выполнения запроса 2:

План 1:



План 2:



Для второго запроса есть два возможных плана выполнения: с фильтрацией после соединения (план 1) и с фильтрацией до соединения (план 2). Оптимальным является второй вариант, поскольку фильтрация до соединения существенно уменьшает объем данных, участвующих в операциях соединения, и ускоряет выполнение запроса.

После добавления индексов план выполнения запроса изменится: СУБД начнет использовать B-tree индексы по полям Н_ЛЮДИ.ОТЧЕСТВО и Н_ВЕДОМОСТИ.ДАТА. Это позволит сразу выбирать только те строки из таблицы Н_ЛЮДИ, у которых отчество меньше 'Владимирович', и только те строки из таблицы Н_ВЕДОМОСТИ, у которых дата меньше '2010-06-18', без необходимости последовательного просмотра всей таблицы. Таким образом, индексы позволяют реализовать оптимизированный план максимально эффективно, обеспечивая быстрый доступ к нужным данным и ускоряя выполнение запроса.

Выполнение EXPLAIN ANALYZE:

```
QUERY PLAN
-----
Nested Loop (cost=0.60..2639.32 rows=77944 width=16) (actual time=0.418..5.555 rows=5244 loops=1)
  Join Filter: ("Н_ЛЮДИ"."ИД" = "Н_ВЕДОМОСТИ"."ЧЛВК_ИД")
  -> Nested Loop (cost=0.29..279.41 rows=1360 width=16) (actual time=0.056..2.782 rows=1237 loops=1)
    -> Seq Scan on "Н_СЕССИЯ" (cost=0.00..108.52 rows=3752 width=12) (actual time=0.008..0.404 rows=3752 loops=1)
    -> Memoize (cost=0.29..0.44 rows=1 width=4) (actual time=0.000..0.000 rows=0 loops=3752)
      Cache Key: "Н_СЕССИЯ"."ЧЛВК_ИД"
      Cache Mode: logical
      Hits: 3571 Misses: 181 Evictions: 0 Overflows: 0 Memory Usage: 15kB
      -> Index Scan using "ЧЛВК_РК" on "Н_ЛЮДИ" (cost=0.28..0.43 rows=1 width=4) (actual time=0.003..0.003 rows=0 loops=181)
        Index Cond: ("ИД" = "Н_СЕССИЯ"."ЧЛВК_ИД")
        Filter: (("ОТЧЕСТВО")::text < 'Владимирович')::text
        Rows Removed by Filter: 1
  -> Memoize (cost=0.30..6.20 rows=60 width=8) (actual time=0.000..0.001 rows=4 loops=1237)
    Cache Key: "Н_СЕССИЯ"."ЧЛВК_ИД"
    Cache Mode: logical
    Hits: 1158 Misses: 79 Evictions: 0 Overflows: 0 Memory Usage: 33kB
    -> Index Scan using "ВЕД_ЧЛВК_РК_ИД" on "Н_ВЕДОМОСТИ" (cost=0.29..6.19 rows=60 width=8) (actual time=0.003..0.014 rows=9 loops=79)
      Index Cond: ("ЧЛВК_ИД" = "Н_СЕССИЯ"."ЧЛВК_ИД")
      Filter: ("ДАТА" < '2010-06-18 00:00:00'::timestamp without time zone)

Planning Time: 1.203 ms
Execution Time: 5.896 ms
(21 строка)
```

Вывод:

В процессе выполнения лабораторной работы я познакомился с индексами и оптимизацией запросов.

